Project Koios Optimization
User Manual

Project Koios

Contents

1 alignment **1**
- 1.1 Ownership . . . 1
- 1.2 Available families . . . 1
- 1.3 Evidence and interpretation . . . 1

2 alignment/procrustes **2**
- 2.1 Solver hierarchy . . . 2
- 2.2 Rank behavior . . . 2
- 2.3 Factory use . . . 2

3 alignment/batched_procrustes **3**
- 3.1 Nominal boundary . . . 3
- 3.2 Evidence . . . 3
- 3.3 Fail-closed unique access . . . 3
- 3.4 Empty batches . . . 3

4 NumPy batched Procrustes **4**
- 4.1 Public façade . . . 4
- 4.2 Validation . . . 4
- 4.3 Consumer use . . . 4

5 alignment/batched_procrustes/scipy **5**

Chapter 1

alignment

Alignment selects mathematical transformations between prepared representations. Optimization owns transformation selection and numerical evidence. It does not own the physical or scientific meaning of the representations.

1.1 Ownership

PhysKit may adapt units, operators, lattices, hoppings, and frame identities into unit-free arrays. KSD may select scientific controls and interpret evidence. These consumer responsibilities do not enter Optimization.

1.2 Available families

The library provides one-pair complex-unitary right Procrustes alignment and a dense NumPy batched family. Both act on the right of the source and share one generic numerical policy owner.

1.3 Evidence and interpretation

Rank, residual, cutoff, and backend metadata are software evidence. Consumers must not infer scientific validity from full rank or a small residual alone.

Chapter 2

alignment/procrustes

For finite equally shaped matrices $A, B \in \mathbb{C}^{m \times n}$, scalar Procrustes solves

$$\min_{Q^\dagger Q = I} \|AQ - B\|_F.$$

The cross matrix is $A^\dagger B$ and the selected full-rank transformation is $Q = UV^\dagger$ for $A^\dagger B = U\Sigma V^\dagger$.

2.1 Solver hierarchy

`ComplexUnitaryProcrustesSolver[DefinitionT, ResultT]` owns shared tolerance, rank-cutoff, residual-roundoff, and backend-evidence policy. `ScalarComplexUnitaryProcrustesSolver` owns the final one-pair solve template. `NumpyComplexUnitaryProcrustesSolver` supplies the reviewed dense SVD.

2.2 Rank behavior

The cutoff is $\tau_a + \tau_r \sigma_{\max}$. Singular values must be strictly greater than it to count. A deficient scalar result withholds the backend's arbitrary null-space completion.

2.3 Factory use

```
from projectkoios.optimization.alignment.procrustes import (
    ComplexUnitaryProcrustesDefinition,
    ComplexUnitaryProcrustesMethod,
    ComplexUnitaryProcrustesSolverFactory,
)

definition = ComplexUnitaryProcrustesDefinition(source, target)
solver = ComplexUnitaryProcrustesSolverFactory.create(
    ComplexUnitaryProcrustesMethod.NUMPY_DENSE
)
result = solver.solve(definition)
```

Tuple matrices are the dependency-free scalar representation.

Chapter 3

alignment/batched_procrustes

Batched Procrustes evaluates the same right-action problem for every item in a preserved leading shape. It does not materialize a Python definition or result object per matrix pair.

3.1 Nominal boundary

`BatchedComplexUnitaryProcrustesSolver` specializes the generic policy to exact NumPy batch definitions and results. It owns vectorized additive-cutoff construction. Concrete backends own decomposition execution.

3.2 Evidence

Every item retains a complete singular spectrum, numerical rank, full-rank status, applied rank cutoff, source and target powers, optimal residual, and squared-residual roundoff cutoff. Deficient completions remain diagnostic.

3.3 Fail-closed unique access

`require_unique_right_unitary_transformations()` returns the existing immutable completion array only when every item is full rank. Otherwise it raises with the first deficient C-order index.

3.4 Empty batches

Zero-sized leading dimensions preserve exact output shapes, claim no backend evidence, and invoke no SVD.

Chapter 4

NumPy batched Procrustes

API chapter: `alignment/batched_procrustes/numpy`

The NumPy backend accepts exact `numpy.ndarray` inputs with exact `numpy.complex128` dtype and shape `(..., m, n)`. It performs one public batched `numpy.linalg.svd` call for a nonempty batch.

4.1 Public façade

```
from projectkoios.optimization.alignment.numpy.batched_procrustes import (
    NumpyBatchedComplexUnitaryProcrustesDefinition,
    NumpyBatchedComplexUnitaryProcrustesSolver,
)

definition = NumpyBatchedComplexUnitaryProcrustesDefinition(
    source_matrices=source,
    target_matrices=target,
)
result = NumpyBatchedComplexUnitaryProcrustesSolver().solve(definition)
transformations = result.require_unique_right_unitary_transformations()
```

4.2 Validation

The solver validates SVD factor unitarity, cross reconstruction, completion unitarity, and direct residual agreement. Nonfinite arithmetic fails closed. A failed batch returns no partial result.

4.3 Consumer use

Consumers preserve leading-index topology and interpret rank and residual evidence after the solve. KSD topology policy and PhysKit physical semantics remain outside this API.

Chapter 5

alignment/batched_procrustes/scipy

No SciPy batched Procrustes solver is currently implemented or exported.

Dense exact complex128 batches use the supported NumPy façade. A future SciPy backend requires a distinct representation or execution need, an explicit contract, independent numerical qualification, and isolated-wheel verification. It must not be introduced as a name-only wrapper around NumPy.

Consumers must not infer SciPy availability from unrelated SciPy capabilities in the distribution.

Verification scope

Software verification and bounded numerical verification establish implementation properties only. They do not establish scientific validity, material-model adequacy, or human acceptance.